

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)**

Факультет компьютерных технологий и прикладной математики

Кафедра информационных технологий

ОТЧЕТ

о выполнении лабораторной работы № 2

по дисциплине «Программная инженерия»

Выполнила: ст. гр. МО 32/2

Переузник В. С.

Проверил: доцент каф. ИТ

Полетайкин А. Н.

Краснодар

2026

Тема: проектирование функциональной структуры ПС.

Индивидуальная тема: интеллектуальная программная система для решения математических задач.

Цель: освоение методики разработки и описания функциональной части разрабатываемой ПС, закрепление навыков применения методик функционально-ориентированного и объектно-ориентированного подходов программной инженерии для разработки и описания функциональности ПС.

Задание:

1. Обосновать создание разрабатываемой ПС и ее структурную организацию.
2. Выполнить описание процесса функционирования объекта компьютеризации в условиях ПС.
3. Выделить и описать автоматизированные функции, исполняемые подсистемой.
4. На основе результатов, полученных при выполнении пп. 1–3, построить комплексную функциональную модель разрабатываемой ПС.

Ход выполнения работы:

1. Обоснование создания ПС и ее структурная организация

Анализ текущего процесса решения математических задач на кафедре прикладной математики КубГУ, проведенный в лабораторной работе №1, выявил ряд критических недостатков: высокие трудозатраты, риск арифметических ошибок при ручных расчетах, низкая скорость получения результата, сложность выбора оптимального численного метода и разрозненность данных. Внедрение разрабатываемой интеллектуальной программной системы намного упростит процесс решения математических задач. Ключевые изменения, вносимые ПС:

1. Интеллектуализация выбора метода — вместо интуитивного выбора студентом система предлагает обоснованную рекомендацию на основе анализа класса задачи.
2. Автоматизация вычислений — исключение арифметических ошибок, сокращение времени расчета с часов до секунд.
3. Встроенная верификация — автоматический контроль точности на каждой итерации.

4. Накопление знаний — каждое решенное задание становится частью базы знаний.
5. Унификация оформления — все отчеты формируются по единому стандарту.

Состав подсистем ПС:

1. Подсистема ввода и парсинга задачи.

Отвечает за преобразование текстового представления математической задачи в структурированный формат, пригодный для машинной обработки. Поддерживает два режима ввода: через специализированные формы вкладок (для учебных целей) и через текстовый запрос естественным языком (для нейросетевого интерфейса).

2. Нейросетевая подсистема решения задач.

Реализована на базе TensorFlow.js и предназначена для обработки запросов на естественном языке. Анализирует введенный пользователем текст, выделяет ключевые слова и математические выражения, определяет тип задачи (уравнение, интеграл, ОДУ, СЛАУ) и формирует решение, используя вычислительное ядро системы. Обучается на данных, накопленных в IndexedDB от решений пользователей в основных вкладках.

3. Вычислительный комплекс.

Ядро системы, реализующее численные методы, перечисленные в техническом задании. Включает четыре специализированные подсистемы:

- 3.1. Подсистема решения нелинейных уравнений

Реализует методы:

- Ньютона (касательных)
- Секущих
- Половинного деления
- Простой итерации

- 3.2. Подсистема численного интегрирования

Реализует методы:

- Симпсона
- Трапеций

- Прямоугольников (средних)
- Монте-Карло

3.3. Подсистема решения обыкновенных дифференциальных уравнений

Реализует методы:

- Рунге-Кутты 4-го порядка
- Эйлера

3.4. Подсистема решения систем линейных алгебраических уравнений

Реализует методы:

- Гаусса
- Зейделя
- Якоби

4. Подсистема визуализации.

Обеспечивает наглядное представление процесса и результатов решения. На базе библиотеки Chart.js реализует построение графиков, отображение таблиц итераций.

5. Подсистема хранения данных.

Использует встроенную браузерную NoSQL-базу данных IndexedDB для накопления пар «входные данные → результат» с целью формирования обучающей выборки для нейросети, сохранения истории решений.

Обоснование выбранной структуры:

1. Модульность — каждая подсистема может разрабатываться, тестироваться и модифицироваться независимо, что упрощает сопровождение и развитие ПС.
2. Полнота охвата предметной области — структура включает четыре специализированные вычислительные подсистемы (уравнения, интегралы, ОДУ, СЛАУ), что покрывает основные классы задач, решаемых численными методами в учебном процессе кафедры.
3. Наличие интеллектуального компонента — выделение нейросетевой подсистемы обработки естественно-языковых запросов позволяет реализовать дополнительный интерфейс взаимодействия с пользователем и накапливать данные для обучения.

4. Масштабируемость — добавление нового численного метода требует лишь расширения соответствующей предметной подсистемы без изменения остальных компонентов.
5. Унификация вычислительного ядра — все предметные подсистемы используют общие механизмы вычислений и визуализации, что обеспечивает единообразие результатов и упрощает разработку.
6. Накопление данных для обучения — подсистема хранения в IndexedDB собирает все решенные задачи, создавая основу для последующего обучения и улучшения нейросетевой подсистемы.
7. Клиент-серверная архитектура — все вычисления выполняются на стороне клиента, что обеспечивает автономность работы и отсутствие затрат на серверную инфраструктуру.

2. Описание процесса функционирования объекта автоматизации в условиях ПС

Объект автоматизации — кафедра прикладной математики КубГУ. Рассмотрим, как изменяется деятельность кафедры (студентов, аспирантов, преподавателей) при решении математических задач после внедрения разработанной интеллектуальной программной системы.

Ниже представлено описание выполнения каждой задачи процесса (декомпозиция из рисунка 3 лабораторной работы №1) в условиях функционирования ПС.

Задача А1. Анализ и формализация условия.

Ранее студент вручную переписывал условие и тратил время на осмысление. Теперь он либо открывает нужную вкладку (уравнения, интегралы, ОДУ, СЛАУ) и вводит выражение в специализированную форму, либо использует нейросетевой интерфейс, вводя запрос естественным языком. Система автоматически проверяет синтаксис и структурирует данные для дальнейшей обработки. Преподаватель при этом освобождается от необходимости объяснять элементарные ошибки ввода.

В задаче задействованы: подсистема ввода и парсинга, интерфейс ПС, нейросетевая подсистема (при текстовом запросе).

Задача А2. Планирование решения и выбор метода.

Раньше студент искал метод в учебниках, часто выбирая интуитивно или первый попавшийся. Теперь в каждой вкладке представлен выпадающий список методов, соответствующих данному классу задач. Студент может:

- согласиться с рекомендацией (один клик);
- выбрать метод вручную из выпадающего списка, если хочет попробовать другой;
- просмотреть краткое описание методов.

Выбор студента фиксируется. Если он отклоняет рекомендацию, этот факт сохраняется в IndexedDB для дообучения нейросети.

В задаче задействованы: интерфейс ПС, вычислительный комплекс (для формирования списка методов), подсистема хранения данных (IndexedDB).

Задача А3: Выполнение вычислений

Вместо ручного счета на калькуляторе или написания скриптов вычислительное ядро ПС за секунды выполняет все расчеты выбранным методом, отображая промежуточные значения (таблицы итераций, оценки погрешности). Ошибки, связанные с человеческим фактором, исключаются.

В результате студент видит не только ответ, но и весь ход решения - точные вычисления без арифметических ошибок, полная прозрачность процесса. Преподаватель при проверке видит те же промежуточные значения, что и студент.

В задаче задействованы: вычислительный комплекс (с соответствующей подсистемой — уравнений, интегралов, ОДУ или СЛАУ), подсистема визуализации.

Задача А4: Проверка и верификация результата

Ранее преподаватель вручную перепроверял все выкладки студента, находил ошибки, возвращает работу на доработку. И это занимало много времени. Теперь система автоматически вычисляет невязку, оценивает погрешность. Преподаватель видит готовый результат с объективными числовыми показателями точности. Также результаты верификации отображаются студенту. Если погрешность превышает допустимую, система предлагает сменить начальные данные или метод. Студент может сразу скорректировать решение.

В задаче задействованы: вычислительный комплекс, подсистема визуализации, интерфейс ПС.

Задача А5: Оформление результата

Студенту больше не нужно вручную переписывать решение и чертить графики. ПС формирует структурированный отчет на странице с условием, выбранным методом, таблицей итераций, итоговым ответом и графиком. Одновременно запись сохраняется в IndexedDB для накопления обучающей выборки нейросети.

В задаче задействованы: подсистема визуализации, подсистема хранения данных (IndexedDB), интерфейс ПС.

Структурные элементы и связи между задачами

Участники процесса: студент, преподаватель, интерфейс ПС, подсистема ввода, нейросеть, вычислительный комплекс, визуализация, БД (IndexedDB).

Связи между задачами:

- A1 → A2: структурированное условие задачи (через подсистему ввода)
- A2 → A3: выбранный метод решения (через интерфейс)
- A3 → A4: результаты вычислений (таблицы итераций, погрешности)
- A4 → A3: (при необходимости) запрос на уточнение расчета
- A3, A4 → A5: все данные для отчета (результаты, графики, проверки)
- A5 → IndexedDB: сохранение записи о решении для обучения нейросети
- A5 → пользователь: итоговый отчет на экране

3. Автоматизированные функции ПС

3.1. Функция «Ввод и парсинг задачи»

Входные данные: текстовая строка с математическим выражением.

Выходные данные: структурированная задача (разобранная на части: тип задачи, функция)

Действия человека:

- Вводит условие задачи

Действия программы:

- Проверяет синтаксис введенного выражения
- Выявляет ошибки ввода
- Преобразует текст в структурированное представление
- Передает результат в подсистему выбора метода

3.2. Функция «Выбор метода решения»

Входные данные: тип задачи (уравнение/интеграл/ОДУ/СЛАУ)

Выходные данные: выбранный метод и его параметры

Действия человека:

- Просматривает список доступных методов для данного класса задач
- Выбирает метод из выпадающего списка
- При желании просматривает краткое описание методов

Действия программы:

- Формирует список доступных методов в зависимости от типа задачи
- Отображает методы в интерфейсе
- Фиксирует выбор пользователя
- Сохраняет информацию о выборе в IndexedDB для обучения нейросети

3.3. Функция «Выполнение вычислений»

Входные данные: структурированная задача и выбранный метод с параметрами

Выходные данные: массив итераций с промежуточными значениями, итоговый результат, оценка погрешности

Действия человека:

- Нажимает кнопку «Решить»
- Просматривает результаты вычислений

Действия программы:

- Запускает соответствующий численный метод (Ньютона, Симпсона, Рунге-Кутты, Гаусса и др.)
- Выполняет итерационные вычисления
- Формирует таблицу с промежуточными значениями после каждого шага
- Вычисляет итоговый результат и погрешность
- Передает результаты в подсистему визуализации

3.4. Функция «Визуализация процесса решения»

Входные данные: массив итераций, итоговый результат, исходная функция

Выходные данные: график функции, отображение точек итераций и начальных (входных) данных, таблица итераций в интерфейсе

Действия человека:

- Просматривает график и таблицу
- При необходимости изменяет параметры отображения

Действия программы:

- Строит график исходной функции с помощью Chart.js
- Отмечает на графике точки итераций
- Формирует HTML-таблицу с итерациями
- Динамически обновляет визуализацию при изменении параметров

3.5. Функция «Сохранение данных для обучения»

Входные данные: условие задачи, выбранный метод, результаты решения

Выходные данные: запись в IndexedDB

Действия человека: не требуются (выполняется автоматически)

Действия программы:

- Формирует объект для хранения: {условие_задачи, тип_задачи, выбранный_метод, результат, параметры,}
- Открывает транзакцию с IndexedDB
- Сохраняет запись в соответствующем хранилище

- Обработывает возможные ошибки записи

3.6. Функция «Обработка естественно-языковых запросов» (нейросетевая)

Входные данные: текстовая строка на естественном языке

Выходные данные: результат решения задачи

Действия человека:

- Вводит текстовый запрос в специальное поле
- Нажимает кнопку «Распознать и решить»

Действия программы:

- Выделяет из текста ключевые слова («интеграл», «уравнение», «диффур», «система», «реши») и математические выражения
- Формирует структурированное представление запроса
- Передает данные в нейросеть (TensorFlow.js)
- Нейросеть, обученная на накопленных в IndexedDB данных из всех решенных задач, решает задачу
- Полученный результат отображается пользователю

4. Комплексная функциональная модель разрабатываемой ПС

На основе результатов, полученных в подразделах 1–3, построена комплексная функциональная модель разрабатываемой интеллектуальной программной системы. Модель включает иерархию диаграмм в нотации IDEF0 и набор диаграмм поведения UML.

4.1. Функционально-ориентированный подход (IDEF0)

Контекстная диаграмма представляет автоматизированный процесс решения математических задач как единое целое — «черный ящик» — с определенными входами, выходами, управлением и механизмами. Данная диаграмма, представленная на Рисунке 1, является переосмыслением модели «Черный ящик» из лабораторной работы №1 с учетом автоматизации, вносимой ПС.

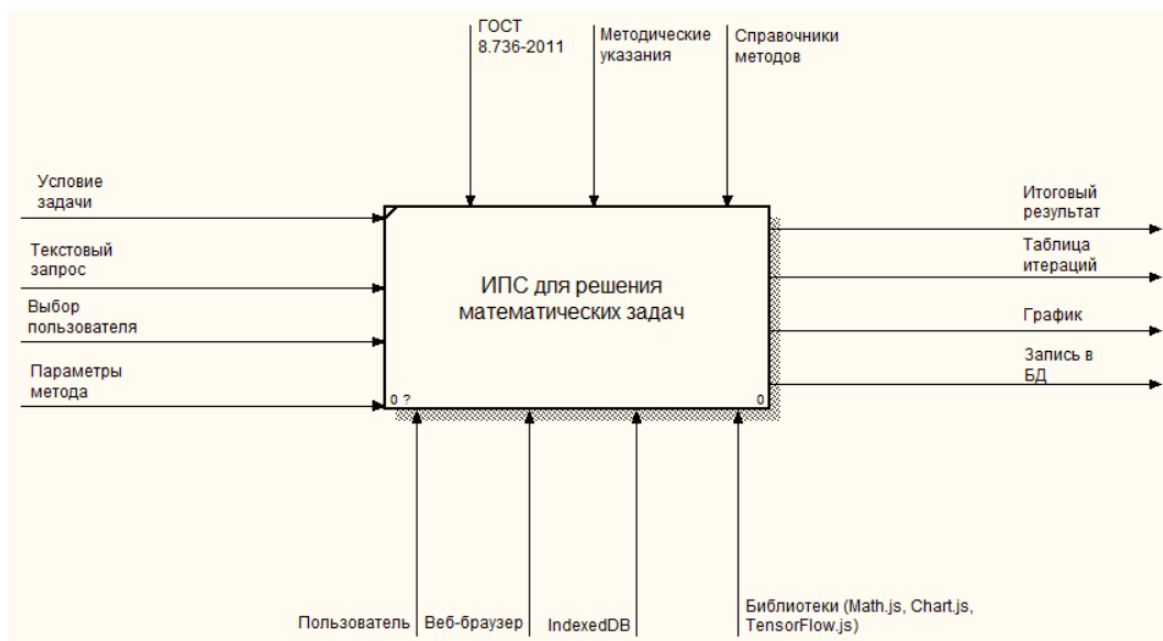


Рисунок 1. Контекстная диаграмма А-0 автоматизированного процесса решения математических задач

На диаграмме декомпозиции А0 (Рисунок 2) контекстная диаграмма раскрывается на пять основных подсистем, соответствующих функциональной структуре ПС, описанной в подразделе 1:

- А1. Подсистема ввода и парсинга задачи — преобразование текстового ввода в структурированные данные
- А2. Нейросетевая подсистема — обработка естественно-языковых запросов, определение типа задачи и рекомендация методов
- А3. Вычислительный комплекс — реализация численных методов для решения уравнений, интегралов, ОДУ и СЛАУ
- А4. Подсистема визуализации — построение графиков и таблиц итераций
- А5. Подсистема хранения данных — сохранение результатов в IndexedDB и загрузка истории для обучения нейросети

Все входные и выходные потоки, управляющие воздействия и механизмы, представленные на контекстной диаграмме А-0, полностью сохранены и распределены между функциональными блоками А1-А5. Новые элементы не вводятся.

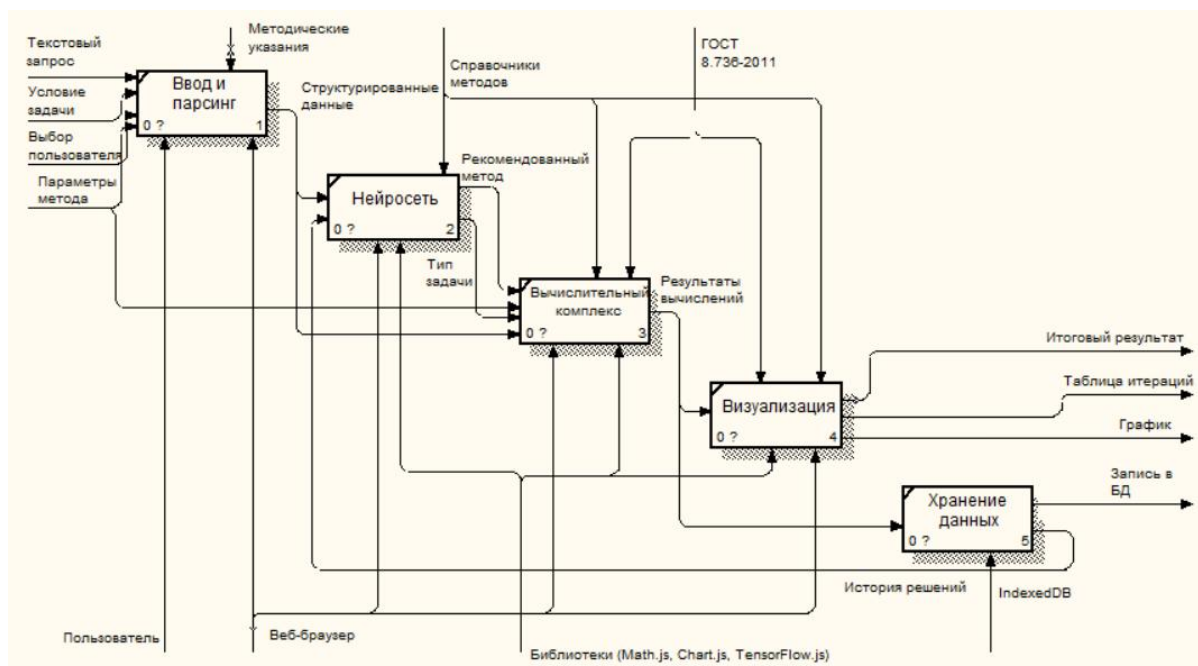


Рисунок 2. Диаграмма декомпозиции A0

Подсистема ввода и парсинга (A1) декомпозируется на три последовательные функции (Рисунок 3):

- A11. Ввод данных — получение данных от пользователя (условие задачи, текстовый запрос, выбор пользователя, параметры метода)
- A12. Валидация и нормализация ввода — проверка синтаксиса математических выражений, выделение ключевых слов из текста
- A13. Структурирование — преобразование проверенных данных в единую структуру для передачи в другие подсистемы

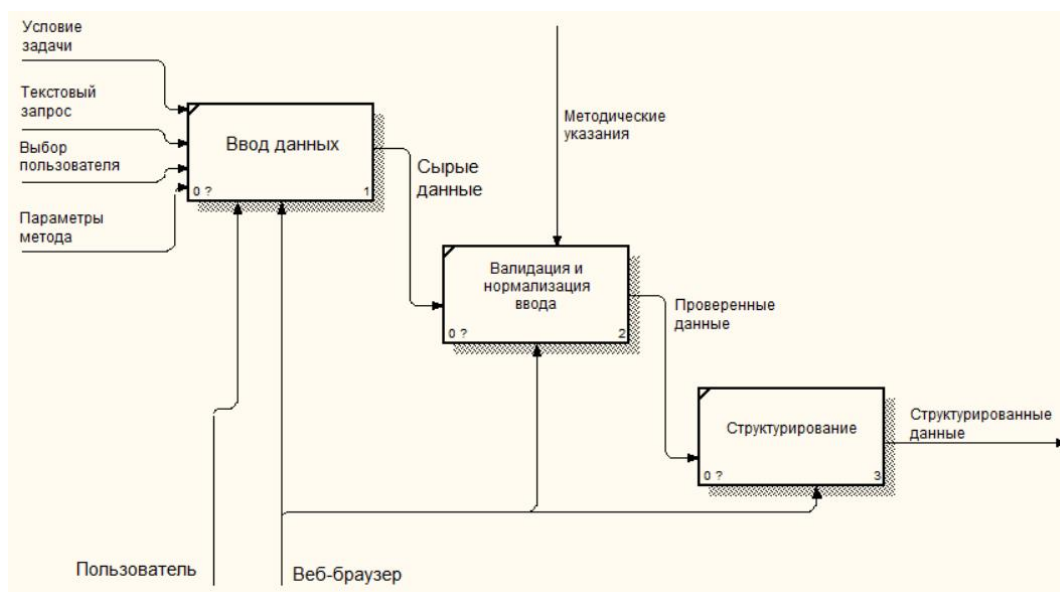


Рисунок 3. Диаграмма декомпозиции A1 «Ввод и парсинг»

Нейросетевая подсистема (A2) декомпозируется на три функции (Рисунок 4):

- A21. Распознавание текста — анализ структурированных данных от A1, выделение ключевых слов
- A22. Определение типа задачи — классификация задачи (уравнение/интеграл/ОДУ/СЛАУ) на основе ключевых слов и истории решений
- A23. Рекомендация метода — подбор оптимального численного метода на основе типа задачи и накопленной статистики

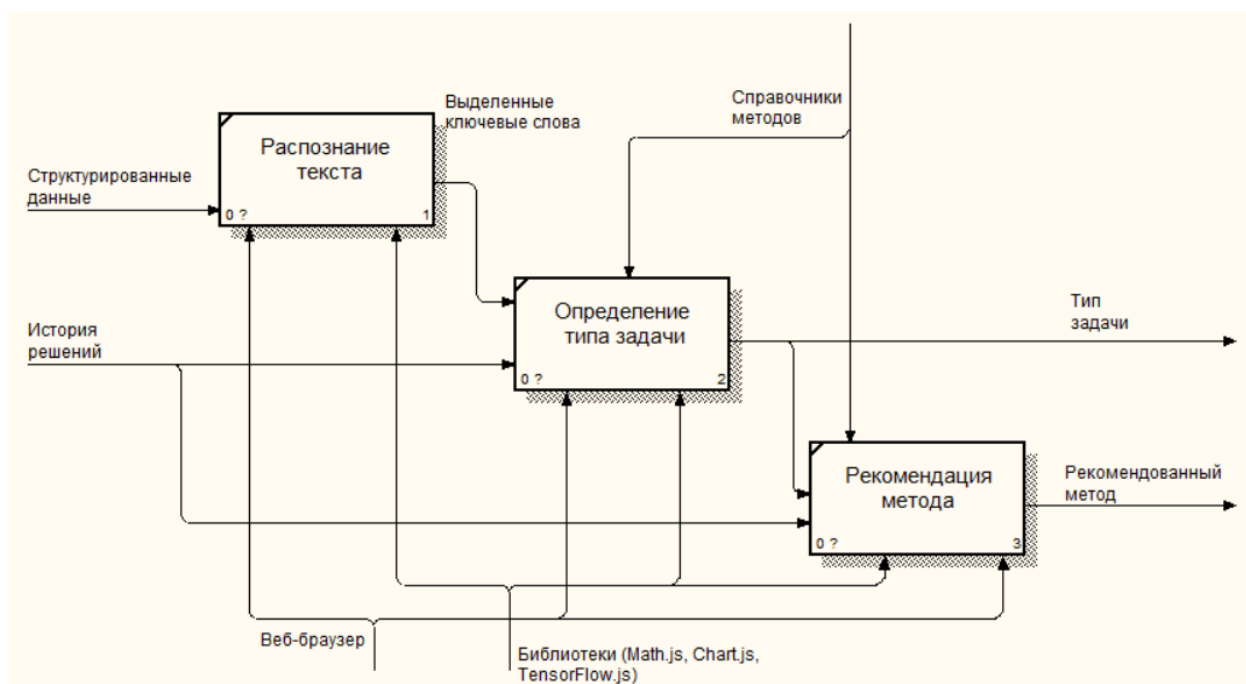


Рисунок 4. Диаграмма декомпозиции A2 «Нейросеть»

Вычислительный комплекс (A3) декомпозируется на четыре специализированные подсистемы по классам задач (Рисунок 5):

- A31. Подсистема решения уравнений — реализует методы Ньютона, секущих, половинного деления, простой итерации
- A32. Подсистема численного интегрирования — реализует методы Симпсона, трапеций, прямоугольников, Монте-Карло
- A33. Подсистема решения ОДУ — реализует методы Эйлера и Рунге-Кутты 4-го порядка
- A34. Подсистема решения СЛАУ — реализует методы Гаусса, Якоби и Зейделя

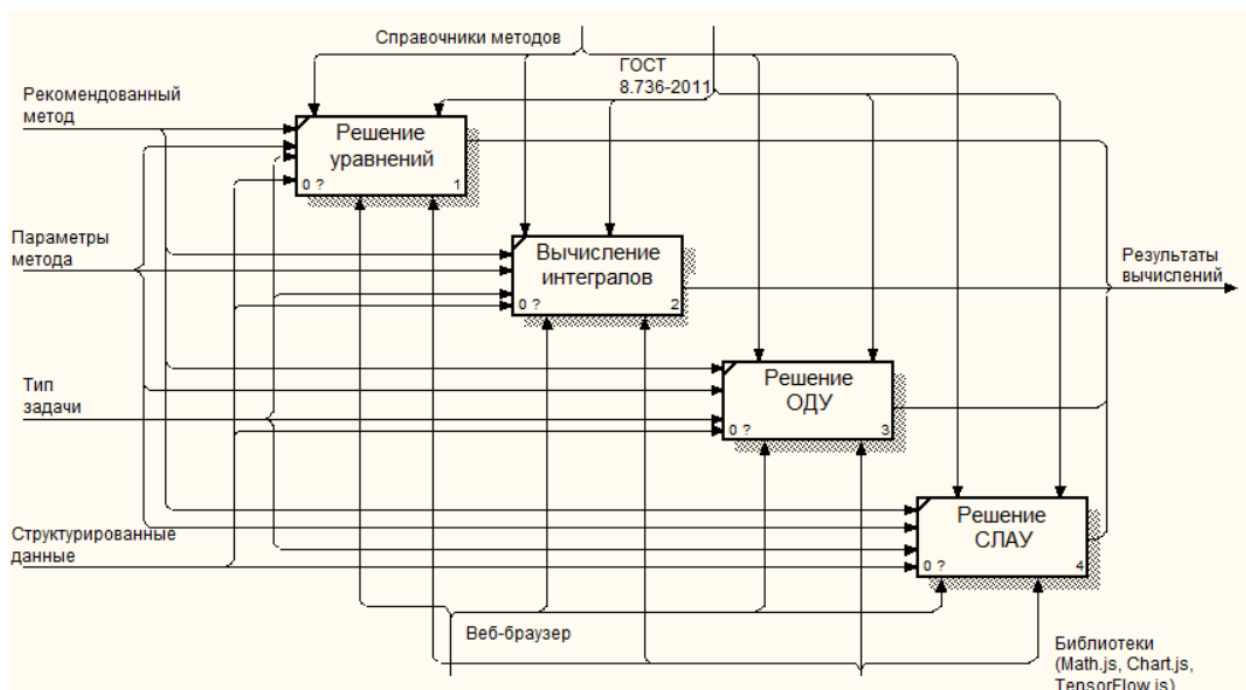


Рисунок 5. Диаграмма декомпозиции АЗ «Вычислительный комплекс»

Подсистема визуализации (А4) декомпозируется на три функции (Рисунок 6):

- А41. Построение графика — создание визуального представления функции и точек итераций с использованием библиотеки Chart.js
- А42. Формирование таблицы — генерация HTML-таблицы с промежуточными значениями итераций методов
- А43. Компоновка отчета — объединение графика, таблицы и итогового результата в единый отчет

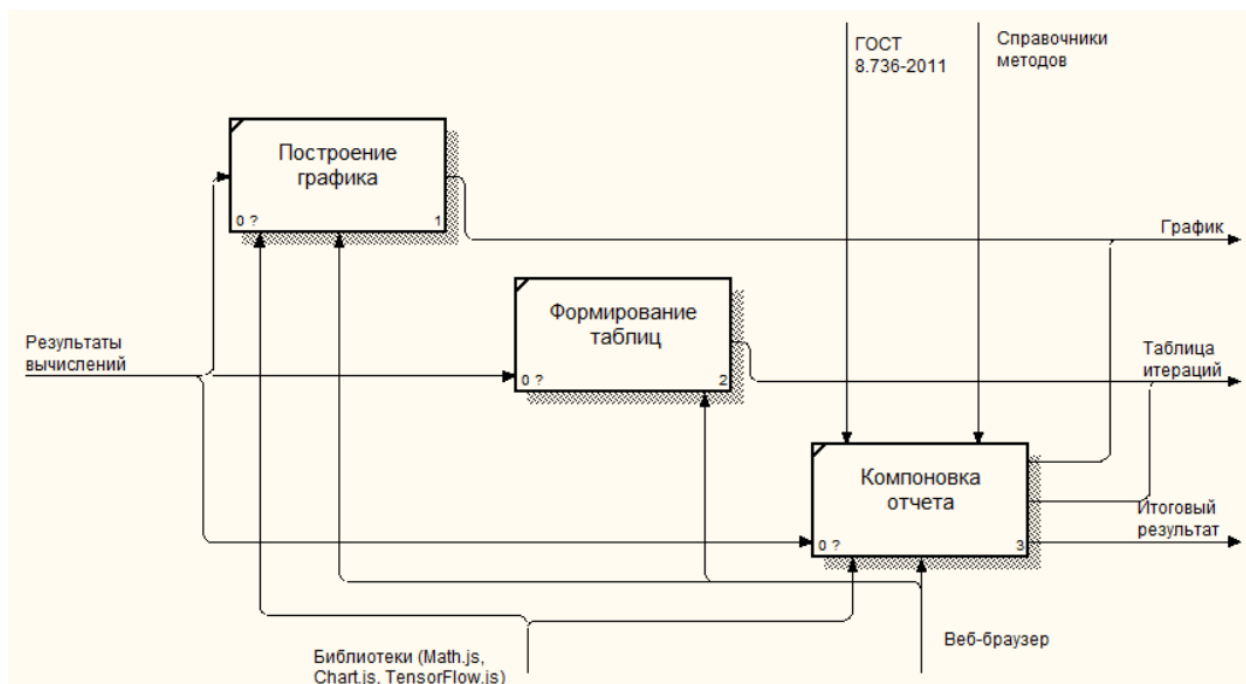


Рисунок 6. Диаграмма декомпозиции A4 «Визуализация»

Подсистема хранения данных (A5) декомпозируется на две функции (Рисунок 7):

- A51. Сохранение результата — запись результатов вычислений в IndexedDB
- A52. Загрузка истории для обучения — извлечение накопленных данных из БД для обучения нейросети

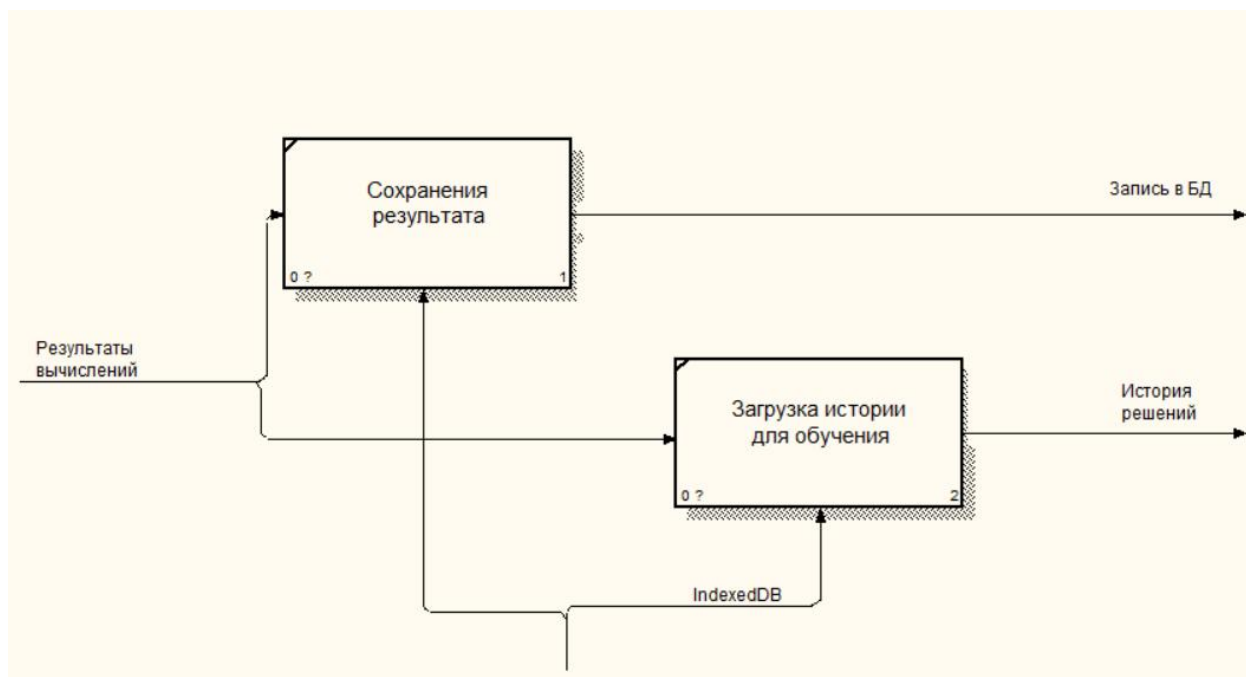


Рисунок 7. Диаграмма декомпозиции A5 «Хранение данных»

Построенная иерархия IDEF0-диаграмм полностью отражает функциональную структуру разрабатываемой ПС. Модель демонстрирует преобразование входных потоков (условие задачи, текстовый запрос, выбор пользователя, параметры метода) в выходные (итоговый результат, таблица итераций, график, запись в БД) под управлением нормативно-справочной документации и с использованием программно-аппаратных механизмов. Каждый уровень декомпозиции сохраняет все внешние интерфейсы вышестоящего уровня, что обеспечивает целостность и непротиворечивость модели.

4.2. Объектно-ориентированный подход (UML)

На диаграмме классов (Рисунок 8) представлены основные классы системы. Актор Пользователь взаимодействует с Менеджером, который координирует работу всех компонентов. Парсер преобразует введенный текст в структурированные данные. Четыре класса (Уравнения, Интегралы, Диффуры, Системы) реализуют соответствующие численные методы. Визуализация отвечает за отображение графиков и таблиц. Нейросеть обрабатывает естественно-языковые запросы, а БД сохраняет историю решений.

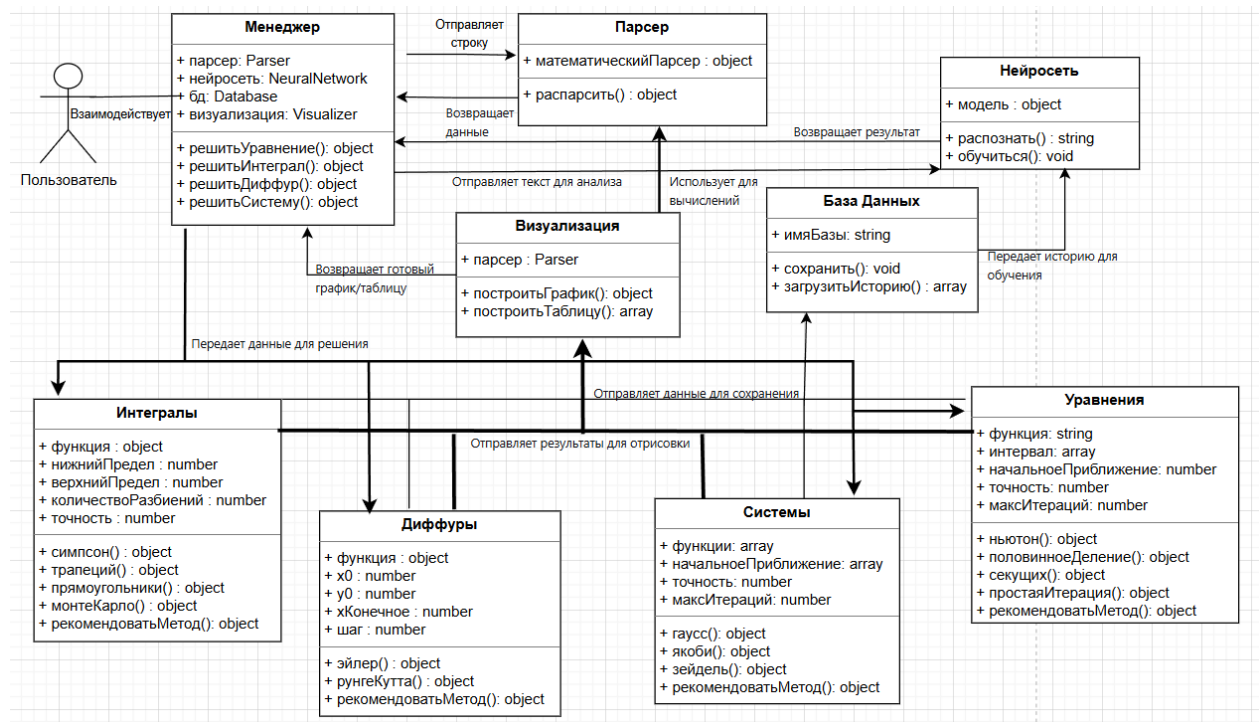


Рисунок 8. Диаграмма классов интеллектуальной ПС

Диаграмма последовательности (Рисунок 9) показывает взаимодействие объектов при решении математической задачи (выполнение основного сценария). Пользователь вводит данные, которые передаются Менеджеру. Менеджер вызывает Парсер для структурирования ввода. Если ввод был выполнен через AI-помощник, Менеджер обращается к Нейросети для определения типа задачи. Затем вызывается соответствующий метод у вычислительной вкладки (Уравнения, Интегралы, Диффуры или Системы), который выполняет вычисления и возвращает результат. После получения результата Менеджер передает данные в Визуализацию для построения графика и сохраняет результат в Базе данных. Финальный результат отображается пользователю.

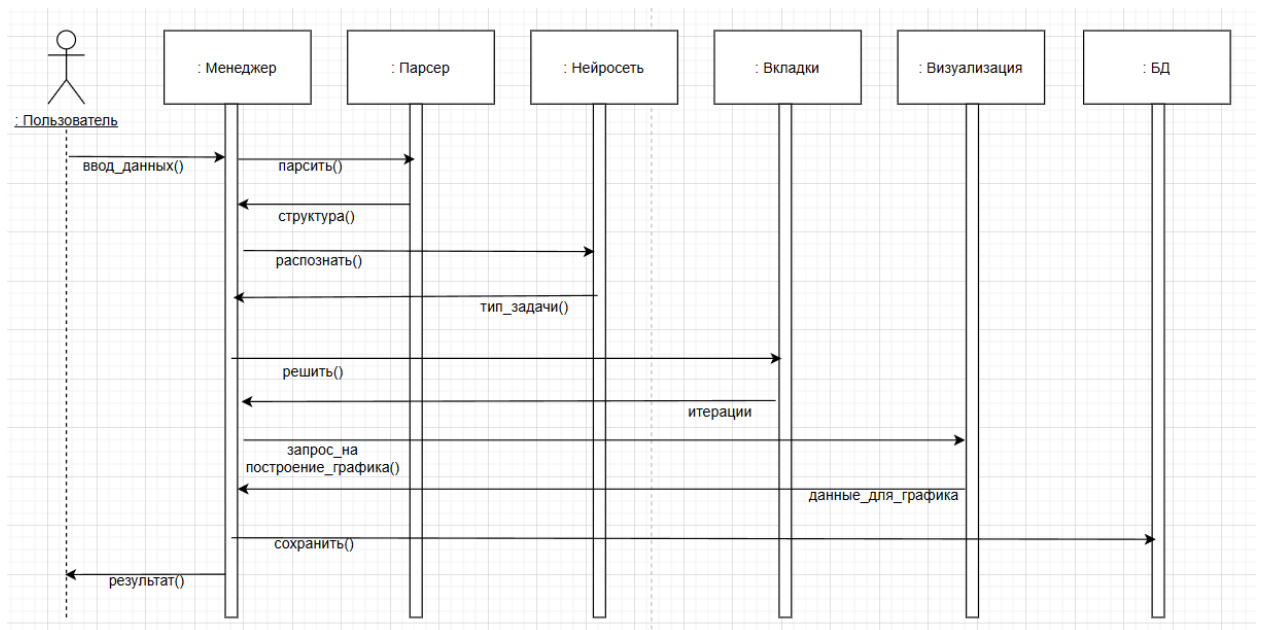


Рисунок 9. Диаграмма последовательности основного процесса

Для классов, характеризующихся нетривиальным поведением и сменой состояний в процессе работы, построены диаграммы состояний.

Диаграмма состояний класса «Менеджер» (Рисунок 10) отражает жизненный цикл управляющего компонента. Начальное состояние — «Ожидание». При вводе данных осуществляется переход в состояние «Парсинг», затем в зависимости от типа ввода — в «Вызов нейросети» или «Вызов метода». После вычислений происходит переход в «Сохранение», а затем либо возврат в «Ожидание», либо в состояние «Ошибка» при возникновении сбоев. Конечными состояниями являются успешное завершение или аварийный останов.

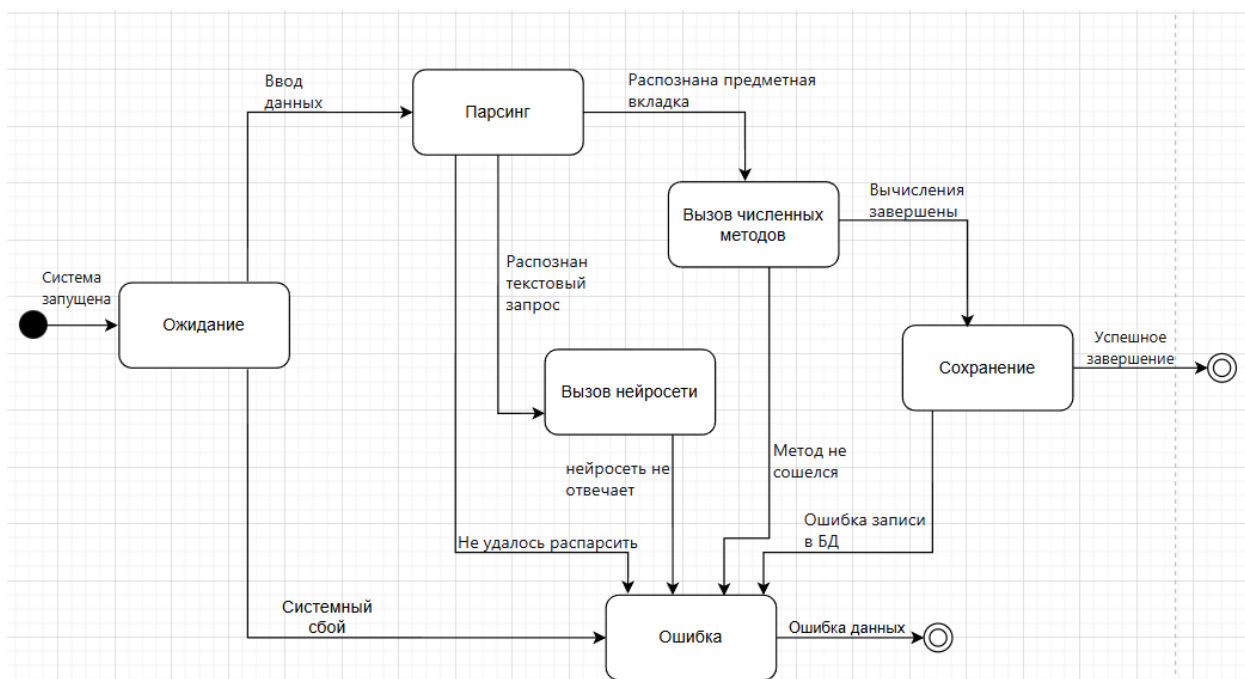


Рисунок 10. Диаграмма состояний класса «Менеджер»

Диаграмма состояний класса «Нейросеть» (Рисунок 11) описывает процесс обучения и работы модели. Изначально нейросеть находится в состоянии «Не обучена». При накоплении достаточного количества данных в БД происходит переход в состояние «Обучение». После успешного обучения модель переходит в состояние «Готова», в котором может распознавать запросы. При поступлении новых данных возможен повторный переход в «Обучение» для дообучения. В случае сбоев предусмотрено состояние «Ошибка».

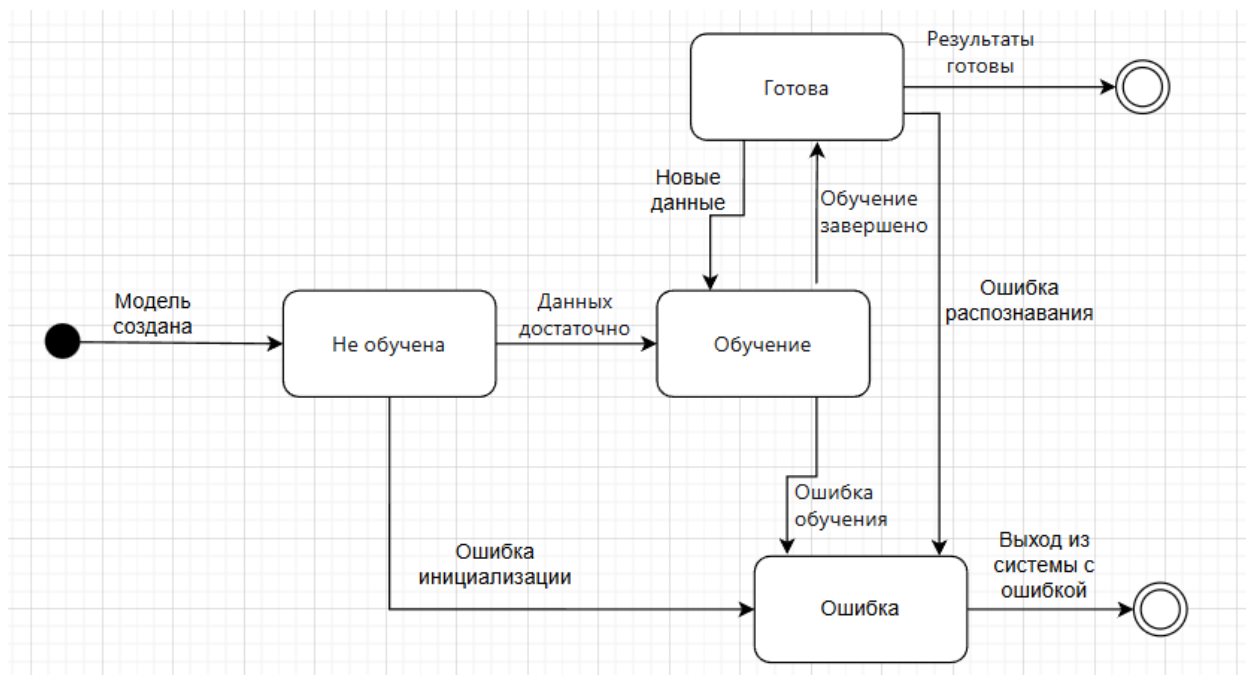


Рисунок 11. Диаграмма состояний класса «Нейросеть»

Диаграмма состояний класса «База данных» (Рисунок 12) показывает состояния работы с IndexedDB. После открытия БД переходит в состояние «Открыта». При поступлении результата вычислений выполняется переход в состояние «Запись», при запросе истории для обучения — в состояние «Чтение». После завершения операций БД возвращается в состояние «Открыта». При закрытии приложения или ошибках доступа предусмотрены конечные состояния.

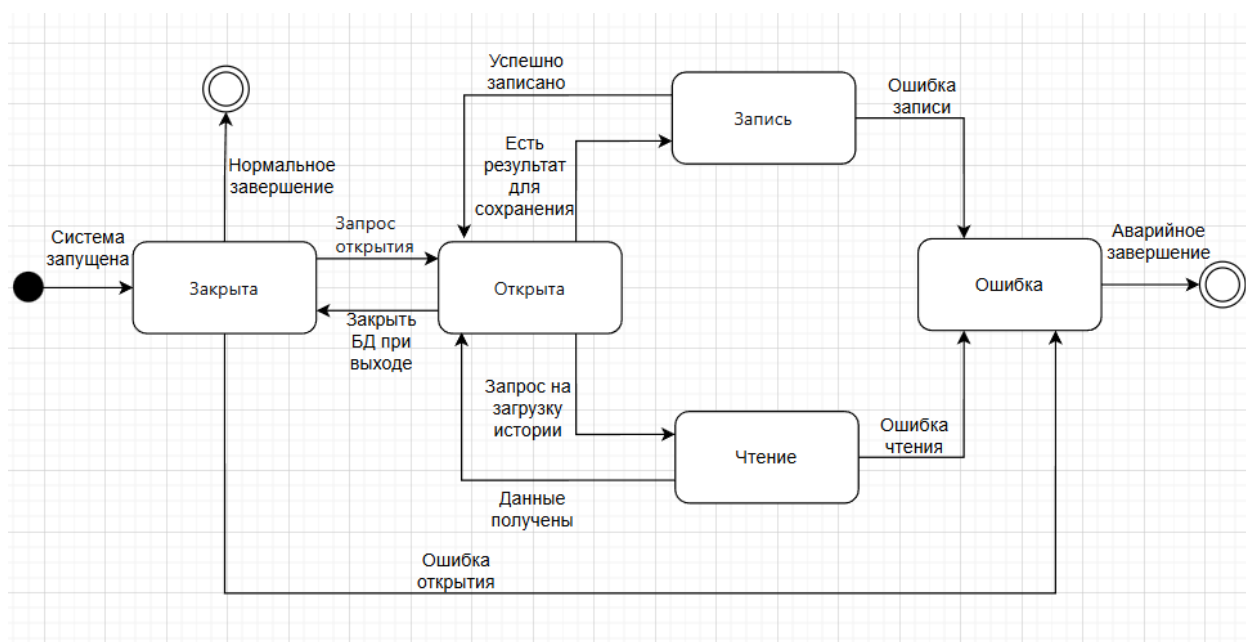


Рисунок 12. Диаграмма состояний класса «База данных»

Диаграмма состояний для обобщенной вычислительной вкладки (Рисунок 13) объединяет поведение классов Уравнения, Интегралы, Диффуры и Системы, так как все они имеют сходный жизненный цикл. После открытия вкладка находится в состоянии «Ожидание ввода». При вводе данных переходит в состояние «Данные получены», затем в «Вычисление». В зависимости от результата вычислений возможны два конечных состояния: «Результат готов» (при успехе) или «Ошибка» (при расхождении/сбое). После отображения результата или сообщения об ошибке работа вкладки завершается.

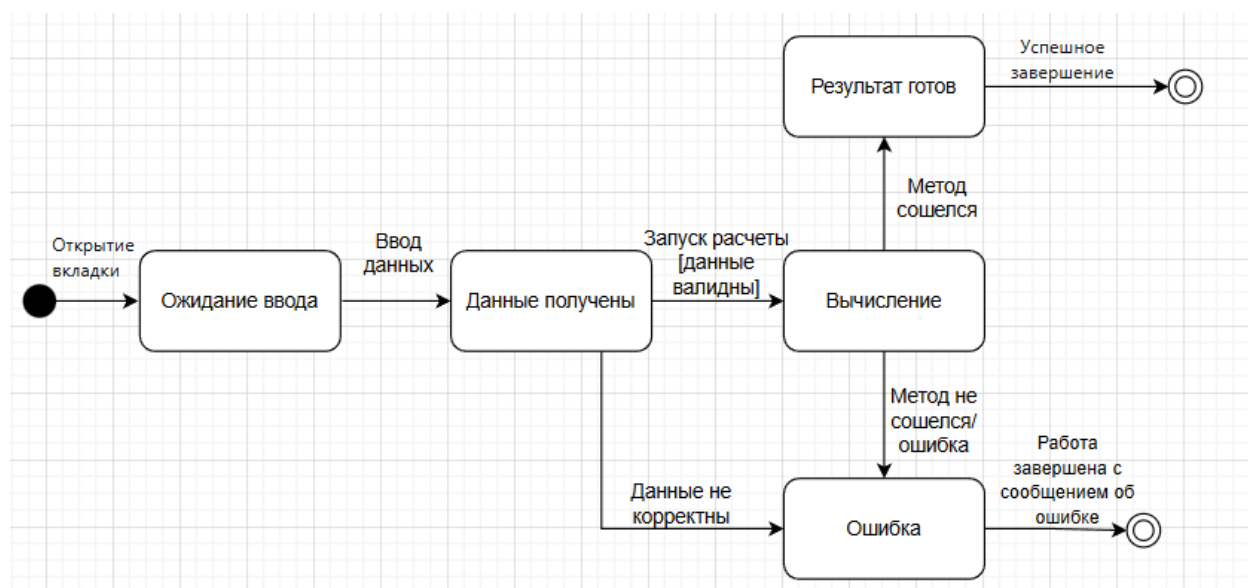


Рисунок 13. Диаграмма состояний для обобщенной вычислительной вкладки

Для функции «Выполнение вычислений» (3.3), характеризующейся наличием циклов и ветвлений, построена диаграмма деятельности (Рисунок 14). Процесс начинается с получения задачи и инициализации параметров. Затем выполняется итерационный цикл: на каждой итерации производится вычисление и проверка сходимости. При достижении сходимости формируется результат и возвращается в менеджер. Если метод не сходится, увеличивается счетчик итераций; при превышении максимального количества формируется сообщение об ошибке.

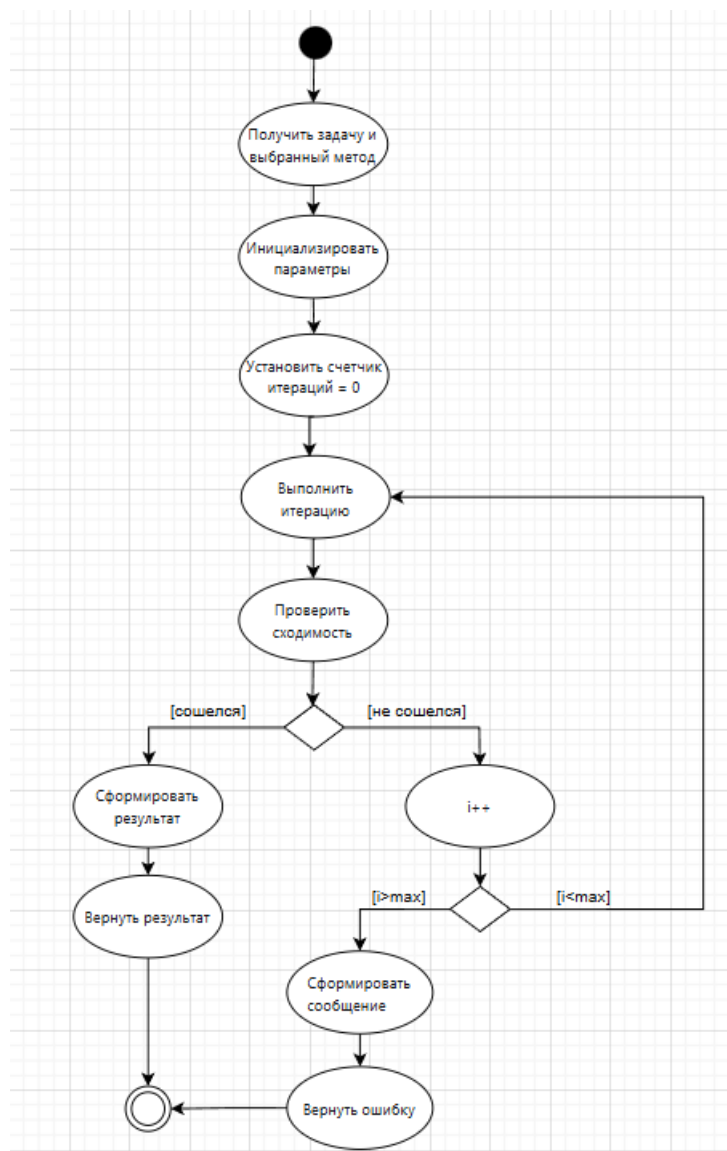


Рисунок 14. Диаграмма деятельности для функции «Выполнение вычислений»

Все построенные диаграммы взаимосвязаны и в совокупности покрывают функциональность, заложенную в диаграмме классов. Диаграмма последовательности демонстрирует взаимодействие объектов при выполнении основных операций, Диаграммы состояний раскрывают поведение ключевых классов при изменении условий, Диаграмма деятельности детализирует алгоритм работы численных методов, показывая циклы и проверки сходимости.

Вывод: в ходе выполнения лабораторной работы была спроектирована функциональная структура интеллектуальной программной системы для решения математических задач. На основе анализа предметной области

выделены автоматизированные функции, построены функциональная модель в нотации IDEF0 и комплекс UML-диаграмм, которые в совокупности полностью описывают структуру и поведение разрабатываемой системы. Полученные результаты являются основой для дальнейшей реализации программного продукта.